

TECH

SYSTEM DESIGN FOR FRONTEND

SYSTEM DESIGN

SYSTEM DESIGN

30 ЗАДАНИЙ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК общая практика индустрии

20 ВОПРОСОВ 7 ДНЕЙ

День 1 ТЯЖЁЛЫЙ	TRADING DASHBOARD POSITIONS			MARKET WATCH ORDER BOOK			P POSITIONS			MARKET WATCH ORDER BOOK			P											
Q1 · Q2 · Q3 · Q4																								
День 2 СРЕДНИЙ	NOTIFICATIONS SIGN SYSTEM			EMPLOYEE PORTAL			DE NOTIFICATIONS SIGN SYSTEM			EMPLOYEE PORTAL			DE											
Q5 · Q6 · Q7																								
День 3 ТЯЖЁЛЫЙ	OBSERVABILITY UTH/PERMS			RELEASE PIPELINE			A OBSERVABILITY UTH/PERMS			RELEASE PIPELINE			A											
Q8 · Q9 · Q10																								
День 4 ТЯЖЁЛЫЙ	MULTI-TENANT NE PWA			FEATURE FLAGS			OFFLI MULTI-TENANT NE PWA			FEATURE FLAGS			OFFLI											
Q11 · Q12 · Q13																								
День 5 ТЯЖЁЛЫЙ	BIG TABLE ENGINE			DASHBOARD BUILDER			FORM BIG TABLE ENGINE			DASHBOARD BUILDER			FORM											
Q14 · Q15 · Q16																								
День 6 ТЯЖЁЛЫЙ	MIGRATIONS: LEGACY/REDUX→RQ/JS→TS/WP→V ITE/MF/MONOREPO						MIGRATIONS: LEGACY/REDUX→RQ/JS→TS/WP→V ITE/MF/MONOREPO																	
Q17 · Q18 · Q19 · Q20 · Q21 · Q22																								
День 7 СРЕДНИЙ	ERROR HANDLING ANALYTICS			CACHE AUTH			i18n WS			A/B ROLLBAC			ERROR HANDLING ANALYTICS			CACHE AUTH			i18n WS			A/B ROLLBAC		
K Q23 · Q24 · Q25 · Q26 · Q27 · Q28 · Q29 · Q30																								

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

TRADING DASHBOARD

MARKET WATCH

POSITIONS

ORDER BOOK

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TRADING DASHBOARD

MARKET WATCH

POSITIONS

ORDER BOOK

ВОПРОСЫ ДНЯ

Q01 Спроектируй frontend для real-time trading dashboard.

Q02 Спроектируй Market Watch UI.

Q03 Спроектируй Positions Table с real-time updates.

Q04 Спроектируй Order Book UI.

Спроектируй frontend для real-time trading dashboard.

ОПИСАНИЕ

Архитектура: WS-feed → ingest layer (батчинг + dedup) → store (useSyncExternalStore) → виджеты (виртуализация + мемо). Reconnect с backoff, status-banner, kill-switch для рискованных фич.

КАК РАБОТАЕТ

- 01 WS-singleton + multiplexing channels.
- 02 Ingest layer: батч + dedup.
- 03 Store через useSyncExternalStore.
- 04 Виджеты с мемо + virtualization.
- 05 Reconnect / backoff / status.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые UIs.
- 02 Real-time trading.

ПОДВОДНЫЕ КАМНИ

- 01 setState на каждый tick – render storm.
- 02 Без kill-switch – невозможно отключить рискованную часть.

МОГУТ ДОСПРОСИТЬ

- 01 Как обработать out-of-order?
- 02 Что такое snapshot+delta?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q02

Спроектируй Market Watch UI.

ОПИСАНИЕ

Список инструментов с обновляемыми ценами. Виртуализация, flash-эффект на изменении (transform/opacity), per-row мемо, sort/filter с useDeferredValue, persist user-настройки.

КАК РАБОТАЕТ

- 01 TanStack Virtual для списка.
- 02 CSS-flash composite-only.
- 03 React.мемо per row.
- 04 useDeferredValue для filter.
- 05 localStorage для user-cols.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading apps.
- 02 Crypto-exchanges.

ПОДВОДНЫЕ КАМНИ

- 01 Layout-shift при flash → CLS.
- 02 DOM-update per tick – laggy.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flash without layout-shift?
- 02 Когда canvas вместо DOM?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/cls>

Спроектируй Positions Table с real-time updates.

ОПИСАНИЕ

Таблица позиций с live-PnL. Diff updates по symbol-id, optimistic при close-position, history audit, виртуализация для большого числа открытых позиций.

КАК РАБОТАЕТ

- 01 Diff updates by id.
- 02 Optimistic close-position.
- 03 Audit-log row.
- 04 Виртуализация для масштаба.
- 05 P&L computed на лету.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Brokerage UI.
- 02 Portfolio management.

ПОДВОДНЫЕ КАМНИ

- 01 Без diff – full re-render.
- 02 Без optimistic – laggy close.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое eventually-consistent UI?
- 02 Как обработать failed close?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/table/latest>

ВОПРОС Q04

Спроектируй Order Book UI.**ОПИСАНИЕ**

Bids / asks с обновлением 100+/sec. Snapshot + incremental updates, Map<price, size> для O(1) update, виртуализация только видимых уровней. Иногда canvas/WebGL.

КАК РАБОТАЕТ

- 01 Snapshot + delta WS protocol.
- 02 Map<price, level>.
- 03 Виртуализация $\pm N$ rows around mid.
- 04 Aggregate by price-tick.
- 05 Иногда canvas.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Crypto / forex L2.
- 02 Algo-trading.

ПОДВОДНЫЕ КАМНИ

- 01 DOM на 1000+ updates/sec – kill UI.
- 02 Без aggregate – слишком много шума.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое L2/L3 data?
- 02 Когда canvas лучше?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/canvas-performance>

NOTIFICATIONS

EMPLOYEE PORTAL

DESIGN SYSTEM

ЧТО УЧИМ В ЭТОТ ДЕНЬ

NOTIFICATIONS EMPLOYEE PORTAL DESIGN SYSTEM

ВОПРОСЫ ДНЯ

Q05 Спроектируй notification center.

Q06 Спроектируй employee portal как microfrontend platform.

Q07 Спроектируй design system для нескольких продуктовых команд.

ВОПРОС Q05

Спроектируй notification center.**ОПИСАНИЕ**

Список уведомлений с read/unread, real-time push, optimistic mark-as-read, badge-count, persist between sessions, infinite-scroll history.

КАК РАБОТАЕТ

- 01 WS push для new notifications.
- 02 Optimistic mark-as-read.
- 03 Badge с unread-count.
- 04 react-query infinite-query.
- 05 Persist через localStorage / API.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 B2C apps.
- 02 SaaS-tools.

ПОДВОДНЫЕ КАМНИ

- 01 Race на mark-as-read.
- 02 Badge не sync между tabs.

МОГУТ ДОСПРОСИТЬ

- 01 Чем optimistic полезен?
- 02 Как sync между tabs?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

Спроектируй employee portal как microfrontend platform.

ОПИСАНИЕ

Shell + N remotes (HR, expense, calendar). Module Federation, shared (auth, design-system), независимые deploys, ErrorBoundary вокруг remotes, fallback при load-error.

КАК РАБОТАЕТ

- 01 webpack ModuleFederation.
- 02 Shell + remotes.
- 03 Shared singleton: react, auth, DS.
- 04 ErrorBoundary per remote.
- 05 Independent CI per team.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team enterprise.
- 02 Internal portals.

ПОДВОДНЫЕ КАМНИ

- 01 Version skew shared.
- 02 Один remote ломает shell без EB.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое singleton в MF?
- 02 Чем eager от lazy share отличается?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

В О П Р О С Q 0 7

Спроектируй design system для нескольких продуктовых команд.

ОПИСАНИЕ

Tokens (CSS-vars) → primitives (headless) → component-обёртки → patterns. Versioned npm package, Storybook docs, governance (PR-review), pilot adoption, feedback channels.

КАК РАБОТАЕТ

- 01 Tokens → primitives → components → patterns.
- 02 Versioned npm.
- 03 Storybook docs.
- 04 Governance + RFC.
- 05 Adoption-metrics.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-product orgs.
- 02 B2B SaaS.

ПОДВОДНЫЕ КАМНИ

- 01 DS без adoption – мёртвый.
- 02 Слишком гибкий API → fragmentation.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RFC process?
- 02 Как metric adoption?

ПОЛНАЯ СТАТЬЯ

<https://www.designsystems.com/>

OBSERVABILITY

RELEASE PIPELINE

AUTH/PERMS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

OBSERVABILITY

RELEASE PIPELINE

AUTH/PERMS

ВОПРОСЫ ДНЯ

Q08 Спроектируй frontend observability platform.

Q09 Спроектируй release pipeline для high-risk fintech UI.

Q10 Спроектируй frontend auth/permissions model.

ВОПРОС Q08

Спроектируй frontend observability platform.**ОПИСАНИЕ**

Sentry для errors, RUM для CWV, custom events для business. OpenTelemetry для distributed tracing. Dashboards в Grafana / Datadog. Alerts с правильным escalation.

КАК РАБОТАЕТ

- 01 Errors: Sentry.
- 02 CWV: web-vitals + RUM.
- 03 Custom events.
- 04 OTEL traces.
- 05 Dashboards + alerts.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production observability.
- 02 SRE-команды.

ПОДВОДНЫЕ КАМНИ

- 01 Лог PII = breach.
- 02 Slow alerts → пропуск incidents.

МОГУТ ДОСПРОСИТЬ

- 01 Чем OTEL полезен?
- 02 Что такое golden signals?

ПОЛНАЯ СТАТЬЯ

<https://opentelemetry.io/>

Спроектируй release pipeline для high-risk fintech UI.

ОПИСАНИЕ

Lint → typecheck → unit → integration → e2e smoke → bundle-size → security-audit → build → deploy-staging → manual QA → canary 1% → 10% → 50% → 100%. Kill-switch на любом этапе.

КАК РАБОТАЕТ

- 01 Quality gates: lint/type/test/bundle/audit.
- 02 Build once, deploy many.
- 03 Canary release.
- 04 Manual QA gate.
- 05 Kill-switch.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading / payment UIs.
- 02 Compliance-критичные.

ПОДВОДНЫЕ КАМНИ

- 01 Skip QA gate под давлением.
- 02 Без canary – все-или-ничего rollout.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое canary?
- 02 Чем kill-switch от rollback отличается?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/CanaryRelease.html>

ВОПРОС Q10

Спроектируй frontend auth/permissions model.

ОПИСАНИЕ

Token-based (JWT в httpOnly cookie или secure storage). RBAC permissions в auth-state. <Can> и useCan для UI. Server обязательно проверяет. Refresh-token flow с singleton.

КАК РАБОТАЕТ

- 01 JWT in httpOnly cookie.
- 02 RBAC in auth-store.
- 03 <Can> component / useCan.
- 04 Singleton refresh.
- 05 Server-side enforcement.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой auth-protected app.
- 02 B2B SaaS.

ПОДВОДНЫЕ КАМНИ

- 01 Token in localStorage = XSS-vulnerable.
- 02 Frontend-only check = security hole.

МОГУТ ДОСПРОСИТЬ

- 01 Чем httpOnly от localStorage защищает?
- 02 Что такое RBAC vs ABAC?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-top-ten/>

MULTI-TENANT

FEATURE FLAGS

OFFLINE PWA

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MULTI-TENANT

FEATURE FLAGS

OFFLINE PWA

ВОПРОСЫ ДНЯ

Q11 Спроектируй multi-tenant frontend.

Q12 Спроектируй feature flag system на frontend.

Q13 Спроектируй offline-capable PWA.

ВОПРОС Q11

Спроектируй multi-tenant frontend.

ОПИСАНИЕ

Tenant-scoped data (filter by tenant_id), tenant-specific theming (CSS-vars), feature flags per tenant, isolation в storage (prefix), audit-log per tenant.

КАК РАБОТАЕТ

- 01 Tenant-id в auth context.
- 02 CSS-vars per brand.
- 03 Feature-flags per tenant.
- 04 Storage prefix.
- 05 Per-tenant analytics.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 B2B SaaS.
- 02 White-label.

ПОДВОДНЫЕ КАМНИ

- 01 Tenant cross-leak (cookie / cache).
- 02 Performance: per-tenant configs.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tenant isolation?
- 02 Чем CSS-vars удобны для themes?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/color-scheme>

Спроектируй feature flag system на frontend.

ОПИСАНИЕ

Provider (LaunchDarkly / GrowthBook / custom). Загружаются на старте. useFlag-хук, <Feature flag="...">. Server-side evaluation для critical. SSR pre-render с правильными flags.

КАК РАБОТАЕТ

- 01 Provider integration.
- 02 Init on app start.
- 03 useFlag / <Feature> components.
- 04 Server-side eval для critical.
- 05 Tested with both flag values.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 A/B testing.
- 02 Gradual rollout.
- 03 Kill-switch.

ПОДВОДНЫЕ КАМНИ

- 01 String-keys без union – опечатки.
- 02 Forgotten flags = долг.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое gradual rollout?
- 02 Как тестировать обе ветки?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/>

В О П Р О С Q 13

Спроектируй offline-capable PWA.**ОПИСАНИЕ**

Service Worker с precache (статика) + runtime cache (API). Background Sync для отложенной отправки. IndexedDB для local-first data. Offline-banner. Update-flow без bad UX.

КАК РАБОТАЕТ

- 01 SW с precache + runtime.
- 02 Background Sync для writes.
- 03 IndexedDB для local-data.
- 04 Offline banner.
- 05 Update flow с user-controlled reload.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile-friendly tools.
- 02 Field-service apps.

ПОДВОДНЫЕ КАМНИ

- 01 Stale data без указания.
- 02 Conflict resolution при sync.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Background Sync?
- 02 Чем local-first полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/explore/progressive-web-apps>

BIG TABLE

DASHBOARD BUILDER

FORM ENGINE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BIG TABLE DASHBOARD BUILDER FORM ENGINE

ВОПРОСЫ ДНЯ

Q14 Спроектируй large data table с sorting/filtering/pagination.

Q15 Спроектируй dashboard builder.

Q16 Спроектируй form engine.

ВОПРОС Q14

Спроектируй large data table с sorting/filtering/pagination.

ОПИСАНИЕ

TanStack Table (логика) + Virtual (rendering). Server-side sort/filter/page для огромных datasets. URL-state. React.memo per row. Skeleton при loading. Row-actions с permissions.

КАК РАБОТАЕТ

- 01 TanStack Table + Virtual.
- 02 Server-side sort/filter/page.
- 03 URL: ?sort=&filter=&page=.
- 04 Per-row React.memo.
- 05 Permissions-aware actions.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Admin tables.
- 02 Reporting tools.

ПОДВОДНЫЕ КАМНИ

- 01 Client-side sort на 100k rows – kill.
- 02 Без URL – refresh теряет.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое cursor pagination?
- 02 Чем virtualization полезен?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/table/latest>

Спроектируй dashboard builder.

ОПИСАНИЕ

Drag-drop виджетов на grid. Конфиг = JSON, persist в backend. Каждый виджет – feature с своим data-source. Permissions per dashboard. Share-via-URL.

КАК РАБОТАЕТ

- 01 React DnD / dnd-kit.
- 02 JSON-схема layout.
- 03 Виджет = lazy-loaded компонент.
- 04 Permissions per dashboard.
- 05 Share-link генерация.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Analytics tools.
- 02 BI-platforms.

ПОДВОДНЫЕ КАМНИ

- 01 Без schema-versioning – break при evolution.
- 02 Permissions сложные на shared dashboards.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое JSON-Schema versioning?
- 02 Как share с permissions?

ПОЛНАЯ СТАТЬЯ

<https://dndkit.com/>

В О П Р О С Q 1 6

Спроектируй form engine.

ОПИСАНИЕ

Schema-driven (Zod / JSON-Schema). Field-renderer mapping types → components. Validation + conditional fields через config. Multi-step support. Шаблоны через preset.

КАК РАБОТАЕТ

- 01 Schema-driven (Zod / JSON-Schema).
- 02 Renderer mapping.
- 03 Conditional logic in schema.
- 04 Multi-step через FSM.
- 05 Preset templates.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 No-code form builders.
- 02 Survey tools.
- 03 Onboarding flows.

ПОДВОДНЫЕ КАМНИ

- 01 Schema без миграций – break при изменении.
- 02 Conditional fields сложно тестировать.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое JSON-Schema?
- 02 Как версионировать?

ПОЛНАЯ СТАТЬЯ

<https://json-schema.org/>

MIGRATIONS: LEGACY/REDUX→RQ/JS→TS/WP→VITE/MF/MONOREPO

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MIGRATIONS: LEGACY/REDUX→RQ/JS→TS/WP→VITE/MF/MONOREPO

ВОПРОСЫ ДНЯ

- Q17 Спроектируй migration с legacy SPA на новую архитектуру.
- Q18 Спроектируй migration с Redux на React Query + local state.
- Q19 Спроектируй migration с JS на TypeScript.
- Q20 Спроектируй migration с Webpack на Vite.
- Q21 Спроектируй migration на microfrontends.
- Q22 Спроектируй shared package strategy в monorepo.

ВОПРОС Q17

Спроектируй migration с legacy SPA на новую архитектуру.

ОПИСАНИЕ

Strangler-pattern: новый код пишется по новой архитектуре, legacy постепенно переезжает. Параллельная работа (feature flag-based routing). Метрика – % new architecture.

КАК РАБОТАЕТ

- 01 Strangler-pattern.
- 02 Feature-flag routing.
- 03 Pilot одна феча.
- 04 Migration metric tracking.
- 05 Documentation для team.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Brownfield SPA.
- 02 Long-lived products.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang rewrite – обычно убивает.
- 02 Two architectures forever – долг.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое strangler-fig?
- 02 Как balance migration vs delivery?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

Спроектируй migration с Redux на React Query + local state.

ОПИСАНИЕ

Идентифицировать server-state в Redux → переехать в react-query. Client-state остаётся (или Zustand). Постепенно: pilot одна слайс, потом все. Адаптер для interop.

КАК РАБОТАЕТ

- 01 Identify server-state slices.
- 02 Move to react-query.
- 03 Keep client-state в Redux/Zustand.
- 04 Adapter для interop.
- 05 Per-slice migration.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Existing Redux apps.
- 02 Adoption RQ.

ПОДВОДНЫЕ КАМНИ

- 01 Один-в-один rewrite — долго.
- 02 Без cleanup старого Redux — bloat.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое server-state?
- 02 Чем RQ упрощает?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest>

ВОПРОС Q19

Спроектируй migration с JS на TypeScript.

ОПИСАНИЕ

Setup tsconfig allowJs. Convert one file at a time, начиная с листьев. .d.ts для legacy modules. Постепенно strict-флаги. Pilot пакет / module.

КАК РАБОТАЕТ

- 01 allowJs+checkJs=false.
- 02 Convert leaves first.
- 03 .d.ts shims.
- 04 Strict flags incrementally.
- 05 Pilot package.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JS-проекты adopting TS.
- 02 Mature codebases.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang rewrite – утопает.
- 02 any-езде – победили компилятор.

МОГУТ ДОСПРОСИТЬ

- 01 С чего начать?
- 02 Что такое jsdoc-types?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/migrating-from-javascript.html>

Спроектируй migration с Webpack на Vite.

ОПИСАНИЕ

Параллельный setup Vite + сравнение dev-experience. Migrate config (alias, plugins). Compatibility-плагины (legacy). Pilot одна страница / app. Tests + visual regression.

КАК РАБОТАЕТ

- 01 Parallel Vite config.
- 02 Migrate plugins / aliases.
- 03 @vitejs/plugin-legacy если нужно IE.
- 04 Pilot subset.
- 05 Visual regression test.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Vite adoption.
- 02 Performance pain в Webpack dev.

ПОДВОДНЫЕ КАМНИ

- 01 Plugin parity не 100% – workarounds.
- 02 Custom loaders требуют переписать.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Vite быстрее в dev?
- 02 Что такое plugin-legacy?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/why.html>

ВОПРОС Q21

Спроектируй migration на microfrontends.**ОПИСАНИЕ**

Identify boundaries (по командам / domain). Pilot – одна secondary feature как remote. Setup shell + Module Federation. Shared deps (DS, auth). Постепенный extract.

КАК РАБОТАЕТ

- 01 Identify boundaries.
- 02 Setup shell + first remote.
- 03 Shared singleton config.
- 04 ErrorBoundary вокруг.
- 05 Gradual extract per team.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team SPAs.
- 02 Independent-deploy needs.

ПОДВОДНЫЕ КАМНИ

- 01 MF без необходимости – over-engineering.
- 02 Version skew shared.

МОГУТ ДОСПРОСИТЬ

- 01 Когда MF оправдан?
- 02 Чем strangler полезен?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

Спроектируй shared package strategy в monorepo.

ОПИСАНИЕ

pnpm/yarn workspaces. packages/types, packages/ui, packages/utils. Single React version. Versioned releases (Changesets). CI builds affected only (Nx / Turborepo).

КАК РАБОТАЕТ

- 01 pnpm workspaces.
- 02 packages/* по domain.
- 03 Single-version policy.
- 04 Changesets для releases.
- 05 Nx/Turborepo affected builds.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-package frontend orgs.
- 02 Component libraries.

ПОДВОДНЫЕ КАМНИ

- 01 Multiple React copies → broken hooks.
- 02 Без affected builds – slow CI.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое affected builds?
- 02 Чем pnpm hoist полезен?

ПОЛНАЯ СТАТЬЯ

<https://pnpm.io/feature-comparison>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

8 ВОПРОСОВ

ERROR HANDLING

CACHE

i18n

A/B

ANALYTICS

AU

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ERROR HANDLING

CACHE

i18n

A/B

ANALYTICS

AUTH

WS

ROLLBACK

ВОПРОСЫ ДНЯ

Q23 Спроектируй frontend error handling strategy.

Q24 Спроектируй frontend caching strategy.

Q25 Спроектируй i18n architecture.

Q26 Спроектируй A/B testing architecture.

Q27 Спроектируй analytics event tracking.

Q28 Спроектируй secure token handling flow.

Q29 Спроектируй resilient WebSocket client.

Q30 Спроектируй rollback strategy для federated frontend.

Спроектируй frontend error handling strategy.

ОПИСАНИЕ

Layers: try/catch в async, ErrorBoundary в render, global handlers (window error, unhandledrejection). Sentry для отчётов. User-friendly fallback. Retry где возможно.

КАК РАБОТАЕТ

- 01 try/catch в async.
- 02 ErrorBoundary в render.
- 03 window.onerror + unhandledrejection.
- 04 Sentry для отчётов.
- 05 Fallback UI с retry.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production-критичные.
- 02 SaaS-tools.

ПОДВОДНЫЕ КАМНИ

- 01 Boundary в самой себе ломается.
- 02 Async errors не ловятся boundary.

МОГУТ ДОСПРОСИТЬ

- 01 Что boundary НЕ ловит?
- 02 Как обработать chunk-load error?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

В О П Р О С Q 2 4

Спроектируй frontend caching strategy.**ОПИСАНИЕ**

react-query для server-state (с правильными staleTime / cacheTime). Service Worker для статики + offline. URL для view-state. localStorage для preferences. Cache invalidation via mutation onSuccess.

КАК РАБОТАЕТ

- 01 react-query staleTime per query.
- 02 SW: cache-first / SWR / network-first.
- 03 URL для shared state.
- 04 Storage для prefs.
- 05 Mutation invalidation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA с server data.
- 02 PWA.

ПОДВОДНЫЕ КАМНИ

- 01 Cache-everything → stale.
- 02 Cache-nothing → slow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-while-revalidate?
- 02 Чем staleTime от cacheTime отличается?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/important-defaults>

Спроектируй i18n architecture.

ОПИСАНИЕ

react-i18next или FormatJS. Словари в JSON, lazy-load по locale. ICU MessageFormat для plurals/gender. Locale detection (URL > cookie > browser). RTL через dir+CSS logical.

КАК РАБОТАЕТ

- 01 react-i18next / FormatJS.
- 02 JSON dictionaries lazy-loaded.
- 03 ICU MessageFormat.
- 04 Locale detection chain.
- 05 RTL via dir + logical CSS.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-region products.
- 02 Global SaaS.

ПОДВОДНЫЕ КАМНИ

- 01 Hardcoded text везде → нечеловеческий refactor.
- 02 RTL ломает CSS с left/right.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое ICU?
- 02 Чем dir="rtl" полезен?

ПОЛНАЯ СТАТЬЯ

<https://www.i18next.com/>

В О П Р О С Q 2 6

Спроектируй A/B testing architecture.**ОПИСАНИЕ**

Provider (GrowthBook / Optimizely / Statsig). Variant assignment (sticky per user). Analytics tracking exposure + conversion. Statistical significance check. Auto-stop при clear loss.

КАК РАБОТАЕТ

- 01 A/B provider integration.
- 02 Sticky variant per user.
- 03 Track exposure + outcome.
- 04 Stat-significance.
- 05 Auto-stop on clear winner.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Growth-команды.
- 02 Product-experimentation.

ПОДВОДНЫЕ КАМНИ

- 01 Без statistical sign-check – false positives.
- 02 Sticky потеряна → user в разных variants.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stat-sign?
- 02 Чем sticky полезен?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/statistics/overview>

Спроектируй analytics event tracking.

ОПИСАНИЕ

Event-schema (action.entity или semantic). Tracking provider (Amplitude / Mixpanel / Segment). Naming conventions, schema-validation. Без PII. sendBeacon для надёжной доставки.

КАК РАБОТАЕТ

- 01 Event schema (action.entity).
- 02 Provider integration.
- 03 Naming convention enforced.
- 04 Schema validation.
- 05 sendBeacon для unload.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Product analytics.
- 02 Funnel analysis.

ПОДВОДНЫЕ КАМНИ

- 01 Без schema – events плодятся chaotically.
- 02 PII в events = GDPR-issue.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое event schema?
- 02 Чем sendBeacon лучше fetch при unload?

ПОЛНАЯ СТАТЬЯ

<https://posthog.com/docs>

В О П Р О С Q 2 8

Спроектируй secure token handling flow.**ОПИСАНИЕ**

Access-token: in-memory (closure / store). Refresh-token: httpOnly cookie SameSite=Strict. Refresh-flow с singleton. Auto-logout на repeated 401. Не XSS-доступно.

КАК РАБОТАЕТ

- 01 Access in-memory.
- 02 Refresh in httpOnly cookie.
- 03 Singleton refresh-promise.
- 04 Auto-logout.
- 05 CSP против XSS.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой auth-protected app.

ПОДВОДНЫЕ КАМНИ

- 01 LocalStorage = XSS-vulnerable.
- 02 Race на параллельных 401.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое SameSite?
- 02 Чем CSP помогает?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6749>

Спроектируй resilient WebSocket client.

ОПИСАНИЕ

Reconnect с exponential backoff + jitter. Heartbeat для detect stale. Queue messages во время disconnect. Replay при reconnect. Status-events для UI.

КАК РАБОТАЕТ

- 01 Backoff с jitter.
- 02 Heartbeat ping/pong.
- 03 Message queue.
- 04 onReconnect resubscribe.
- 05 Status events: open/close/reconnecting.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time apps.
- 02 IoT / mqtt-bridges.

ПОДВОДНЫЕ КАМНИ

- 01 Без backoff → thundering herd.
- 02 Без heartbeat – silent stale.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое heartbeat?
- 02 Чем jitter защищает?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6455>

ВОПРОС Q30

Спроектируй rollback strategy для federated frontend.**ОПИСАНИЕ**

Versioned remotes (URL с version). Shell может pin старую version remote. Feature-flag для switch версий. Health-check после deploy. Auto-rollback при error spike.

КАК РАБОТАЕТ

- 01 URL: /remote//
- 02 Shell config pin version.
- 03 Feature-flag switch.
- 04 Auto-rollback on error spike.
- 05 Status-banner.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Federated production.
- 02 High-risk releases.

ПОДВОДНЫЕ КАМНИ

- 01 Without versioned URLs – нечего rollback.
- 02 Cache CDN держит старое.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое immutable deploy?
- 02 Как auto-rollback?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- [] Спроектировать trading dashboard / market watch / positions / order book (Q1-Q4)
- [] Спроектировать notifications / employee portal / DS (Q5, Q6, Q7)
- [] Спроектировать observability / release pipeline / auth-perms (Q8, Q9, Q10)
- [] Спроектировать multi-tenant / flags / offline PWA (Q11, Q12, Q13)
- [] Спроектировать big table / dashboard builder / form engine (Q14, Q15, Q16)
- [] Описать миграции legacy/Redux→RQ/JS→TS/WP→Vite/MF/monorepo (Q17-Q22)
- [] Спроектировать error handling / cache / i18n / A/B / analytics / token / WS / rollback (Q23-Q30)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru